

R-Ladies-Melbourne online seminar on

Deep Probabilistic and Generative modeling

Soroor Hedyeh-zadeh
Helmholtz Munich, May 2026



Today's outline

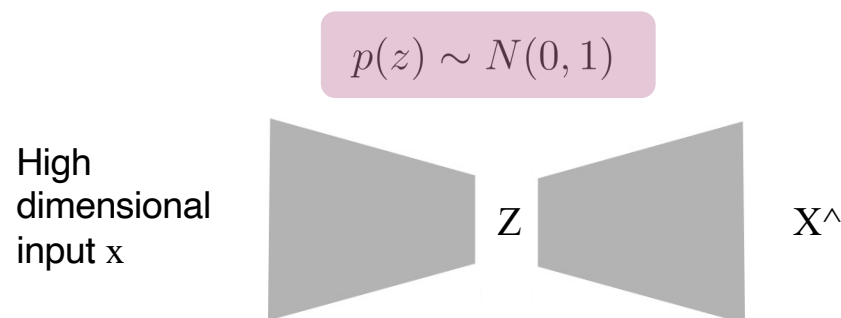
- Variational Autoencoders
- Probabilistic modeling with Variational Inference
- Diffusion Models
- Flow Matching

Generative Modeling| Variational Autoencoders



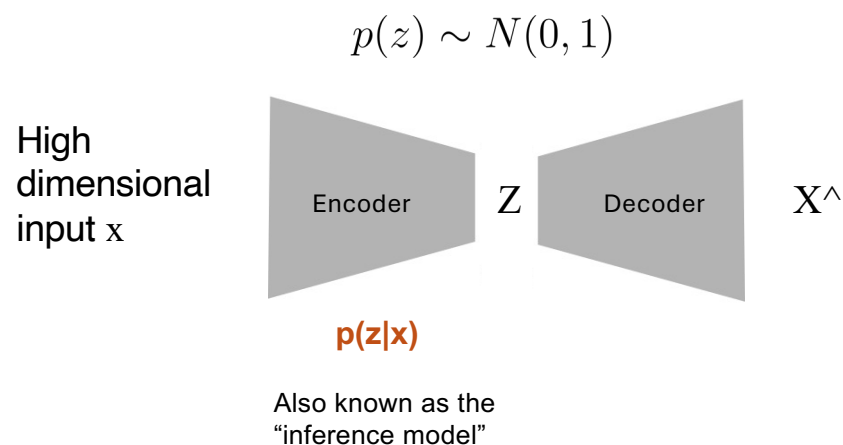
Kingma (2013). Auto-encoding variational bayes. *arXiv*

- Principal Component Analysis is a **linear** dimension reduction. Variational Autoencoders are **non-linear** dimension reduction models.
- Assumption: High-dimensional data are generated by some unknown low-dimensional random variables (**latent variables**). We often denote the latent variables as z . These variables follow a standard normal distribution (**prior**)



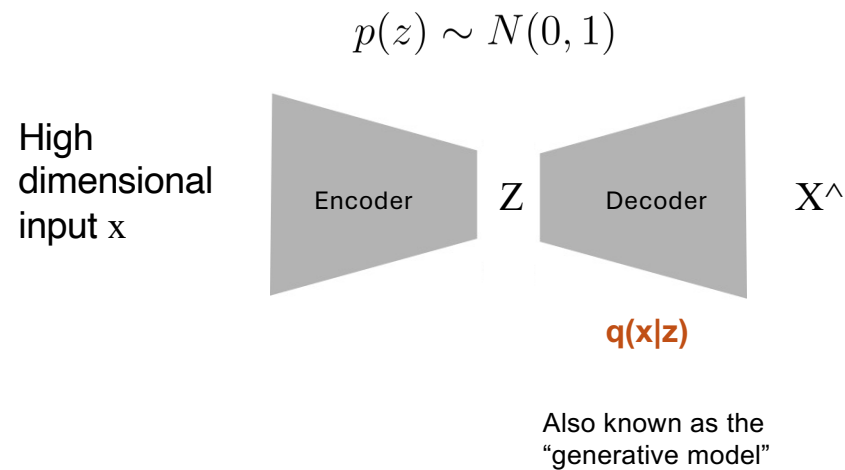
Generative Modeling| Variational Autoencoders

- Assumption: High-dimensional data are generated by some unknown low-dimensional random variables (**latent variables**). We often denote the latent variables as z . These variables follow a standard normal distribution (**prior**)
- Encoder estimates the posterior distribution of the latent variables from the data i.e. **$p(z|x)$**



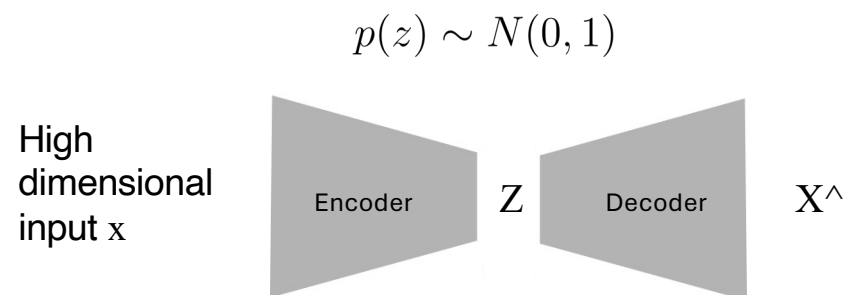
Generative Modeling| Variational Autoencoders

- Assumption: High-dimensional data are generated by some unknown low-dimensional random variables (**latent variables**). We often denote the latent variables as z . These variables follow a standard normal distribution (**prior**)
- Encoder estimates the posterior distribution of the latent variables from the data i.e. **$p(z|x)$**
- Decoder generates the input data from the latent according to a generative process i.e. **$q(x|z)$**



Generative Modeling| Variational Autoencoders

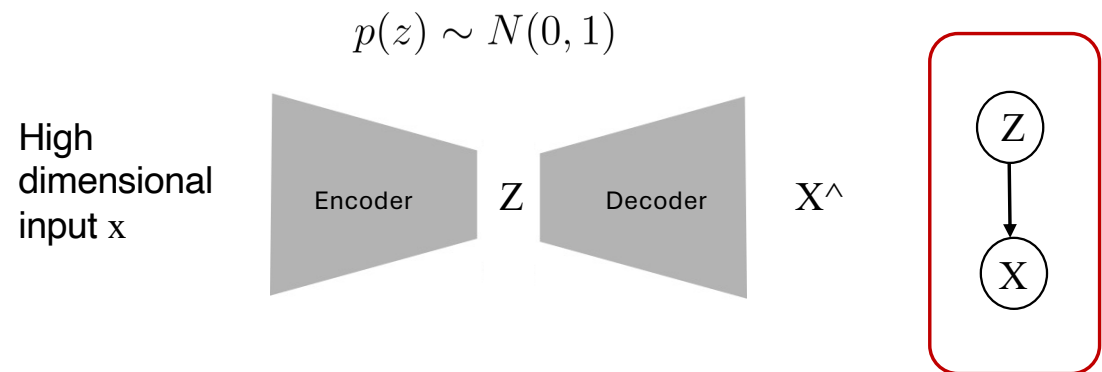
- Assumption: High-dimensional data are generated by some unknown low-dimensional random variables (**latent variables**). We often denote the latent variables as z . These variables follow a standard normal distribution (**prior**)
- Encoder estimates the posterior distribution of the latent variables from the data i.e. **$p(z|x)$**
- Decoder generates the input data from the latent according to a generative process i.e. **$q(x|z)$**
- The network is optimized by minimizing the reconstruction loss (for example Mean Squared Error) and a KL divergence on $p(z)$ for regularization of the latent variables, which make up the **Evidence Lower Bound (ELBO)**



$$\mathcal{L} = \mathbb{E}_{q(z|x)} [\log p(x|z)] - \beta \mathcal{D}_{\text{KL}}[q(z|x) || p(z)]$$

Generative Modeling| Variational Autoencoders

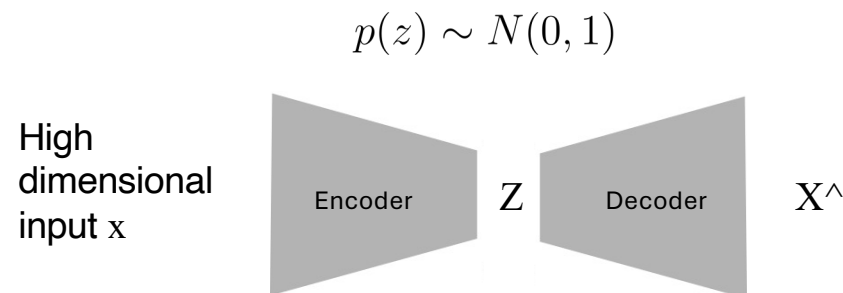
- Assumption: High-dimensional data are generated by some unknown low-dimensional random variables (**latent variables**). We often denote the latent variables as z . These variables follow a standard normal distribution (**prior**)
- Encoder estimates the posterior distribution of the latent variables from the data i.e. $p(z|x)$
- Decoder generates the input data from the latent according to a generative process i.e. $q(x|z)$
- The network is optimized by minimizing the reconstruction loss (for example Mean Squared Error) and a KL divergence on $p(z)$ for regularization of the latent variables, which make up the **Evidence Lower Bound (ELBO)**
- Also represented by Probabilistic Graphical Models (PGMs)



$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \beta \mathcal{D}_{\text{KL}}[q(z|x) || p(z)]$$

Generative Modeling|Variational Autoencoders

- Used for non-linear dimension reduction, denoising and compression
 - Similar training points (e.g. images) on close to each other in the latent space
 - Use latent space for a variety of downstream tasks such as clustering, classification (e.g. via KNN), regression etc
 - The latent variables can also be used as input to other deep learning models



$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \beta \mathcal{D}_{\text{KL}}[q(z|x) || p(z)]$$

Reconstruction loss

KL divergence

set $\beta > 1$ for disentangled VAE

The re-parameterization trick

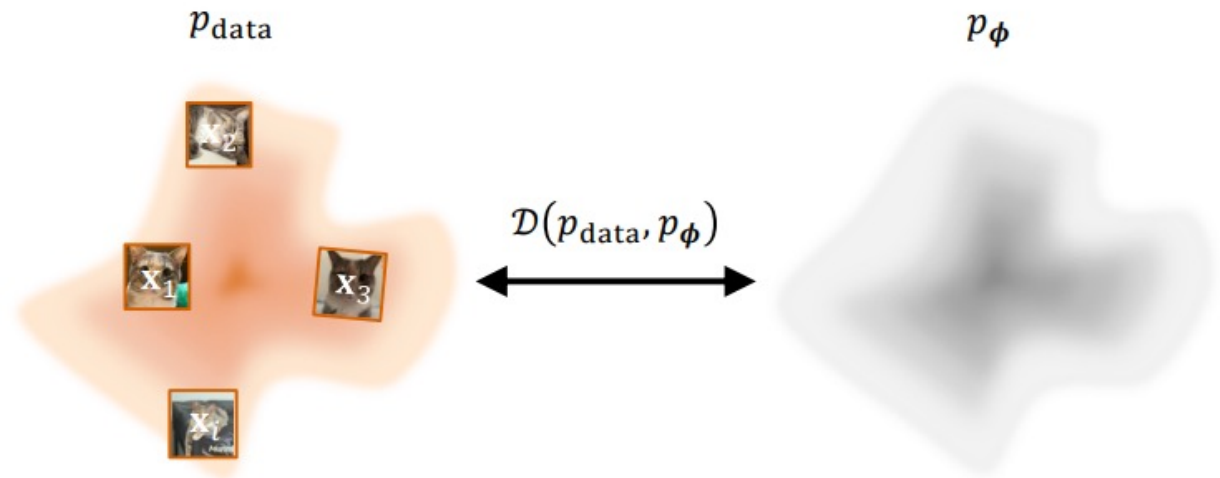
$$q(z | x) = \mathcal{N}(\mu(x), \sigma^2(x))$$

$$z \sim \mathcal{N}(\mu, \sigma^2)$$

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

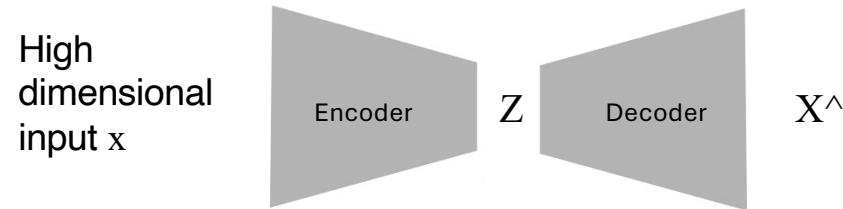
Generative Modeling|Variational Autoencoders

- Used for non-linear dimension reduction, denoising and compression
 - Similar training points (e.g. images) on close to each other in the latent space
 - Use latent space for a variety of downstream tasks such as clustering, classification (e.g. via KNN), regression etc
 - The latent variables can also be used as input to other deep learning models



Generative Modeling| Conditional Variational Autoencoders

- Used for non-linear dimension reduction in **labeled data**, denoising and compression
 - Similar training points (e.g. images) on close to each other in the latent space
 - Use latent space for a variety of downstream tasks such as clustering, classification (e.g. via KNN), regression etc
 - The latent variables can also be used as input to other deep learning models



$$\text{ELBO} = \mathbb{E}_{q(z|x,c)} [\log p(x | z, c)] - D_{\text{KL}} (q(z | x, c) \parallel p(z | c))$$



Sohn (2015). *Learning Structured Output Representation using Deep Conditional Generative Models*.

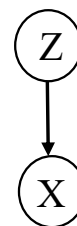
Kingma (2014). *Semi-supervised Learning with Deep Generative Models*.

Generative Modeling| Variational inference

Parameter estimation in a probabilistic model, in which the probabilities are computed by approximation through NN is known as **Variational inference**

Core concepts

- Joint probabilities
- Conditional probabilities
- Using neural networks to estimate the conditional distributions
- More advanced priors



$$p(x, z)$$
$$p(x, z) = p(x|z)p(z)$$

$$p(z) \sim N(0, 1)$$
$$p(x|z) \sim q_{\theta}(x|z)$$

Estimate by Neural
Network

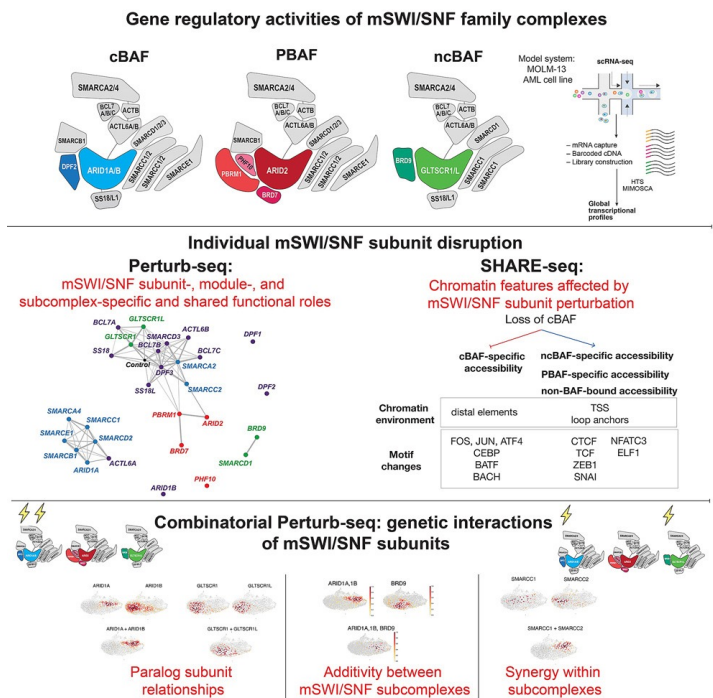


Blei (2017). Variational inference: A review for statisticians.

Generative Modeling| Variational inference

Example: Revisiting the variance model in deep generative models of Perturbseq data

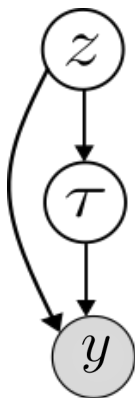
Modeling the variance parameter as a function of a latent variable



$$y \sim EDM(\mu, \phi)$$

$$\text{var}(y) = \phi \mu^b$$

$$\phi_{gc} = \underbrace{\tau_c}_{\text{Cell-specific dispersion (biological)}} + \underbrace{\nu_g}_{\text{gene-specific dispersion (technical)}}$$



$$p(z) \sim N(0, 1)$$

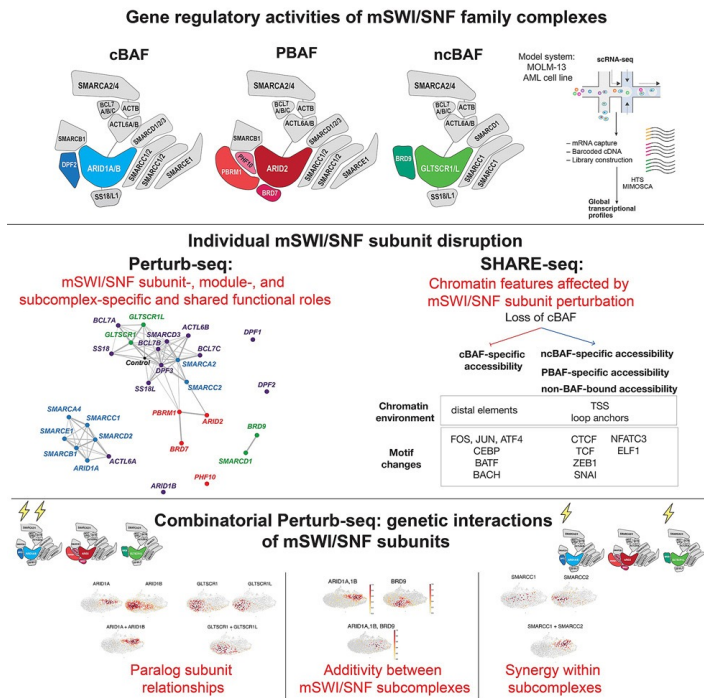
$$p(\tau) \sim N(\mu_{sgRNA}, \sigma_{sgRNA}^2)$$

$$p(z, \tau, y) = p(y|z, \tau)p(t|z)p(z)$$

Approximate using NN
using VAE estimation

Generative Modeling| Variational inference

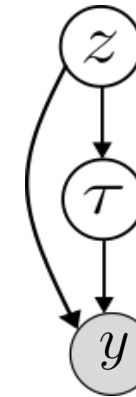
Example: Revisiting the variance model in deep generative models of Perturbseq data



$$y \sim EDM(\mu, \phi)$$

$$\text{var}(y) = \phi \mu^b$$

$$\phi_{gc} = \underbrace{\tau_c}_{\text{Cell-specific dispersion (biological)}} + \underbrace{\nu_g}_{\text{gene-specific dispersion (technical)}}$$



$$p(z) \sim N(0, 1) \quad \text{Note the prior!}$$

$$p(\tau) \sim N(\mu_{sgRNA}, \sigma_{sgRNA}^2)$$

$$p(z, \tau, y) = p(y|z, \tau)p(\tau|z)p(z)$$

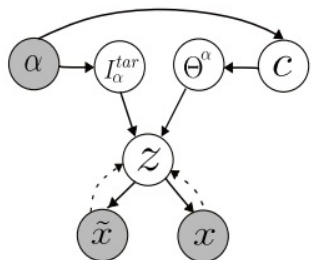
Note need an additional KL term for tau

Variational inference for complex models



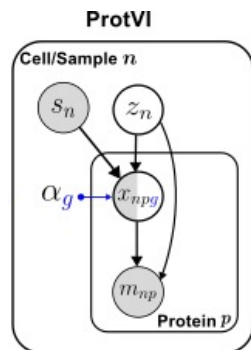
Blei (2017). Variational inference: A review for statisticians.

sVAE-ligr

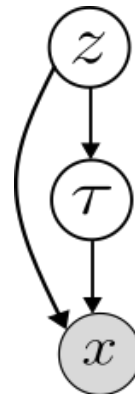


sVAE-ligr data-generating process

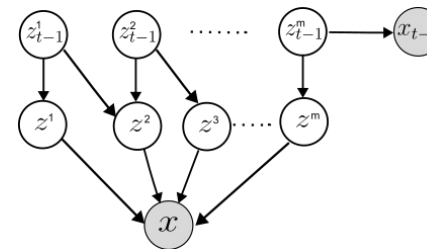
Conditional/hierarchical



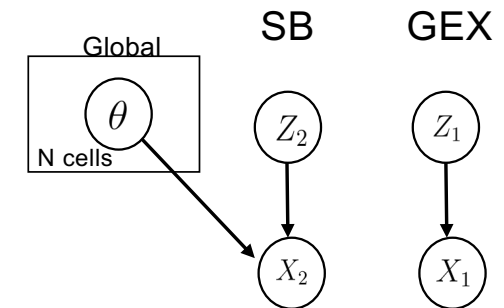
Conditional/hierarchical



Temporal



VI with global params



Disentanglement via Mechanism Sparsity by Replaying Realizations of the Past,
Hediyeh-zadeh, Fischer & Theis,
ICLR 2024 Workshop on Machine Learning for Genomics Explorations.

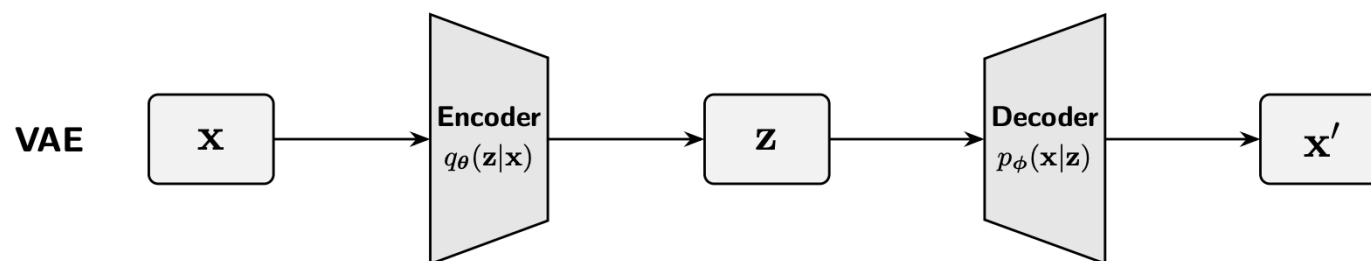
Python libraries



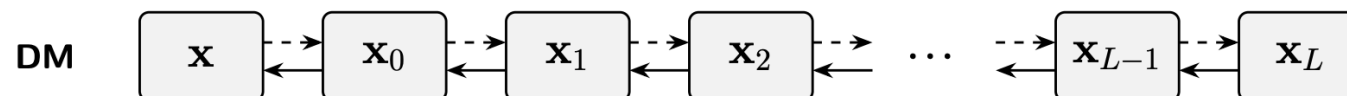
Generative Modeling| Diffusion Models (DM)



Lai (2025). The principles of diffusion models. *arXiv*



Iteratively refines a noisy sample through a multi-step denoising chain



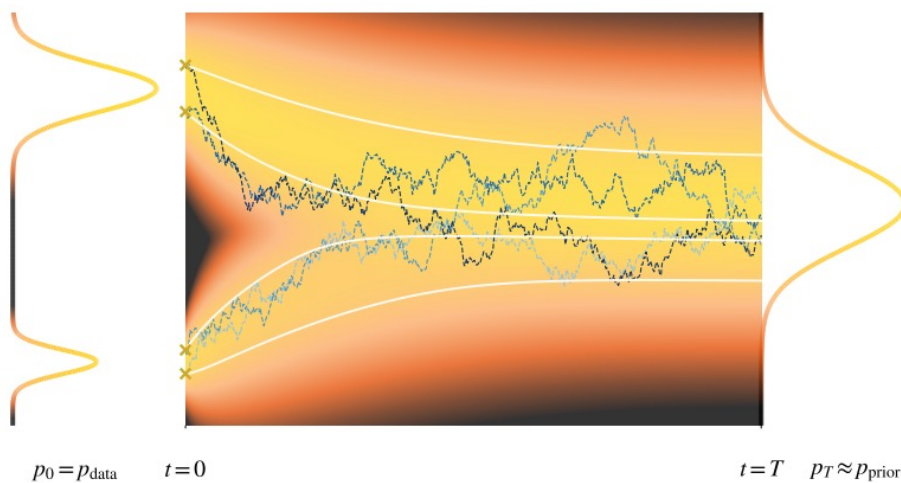
Generative Modeling| Diffusion Models



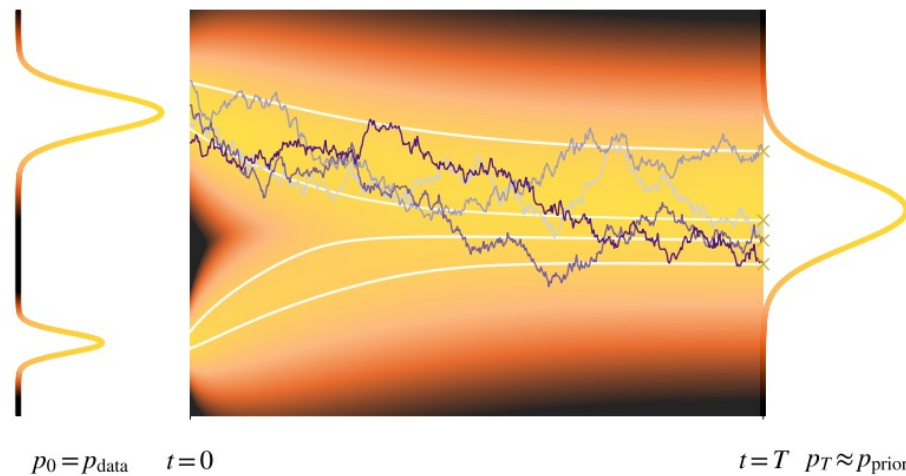
Lai (2025). The principles of diffusion models. *arXiv*

Generation is modeled as denoising the data according to a numerical simulator (ODE/SDE)

Forward process of a diffusion model

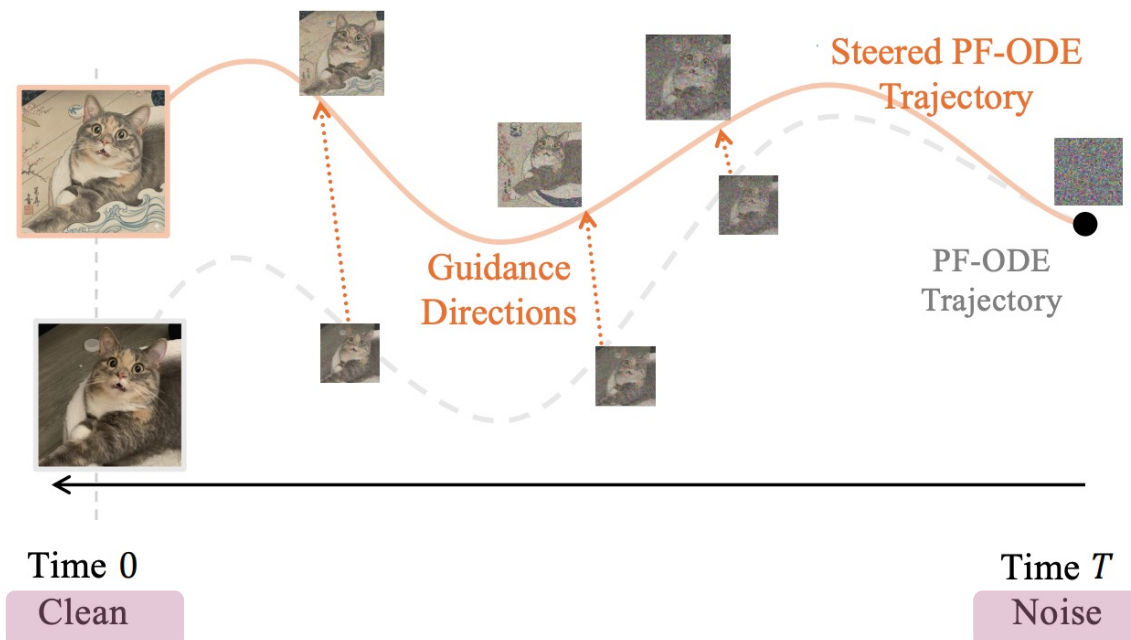


Reverse process of a diffusion model



Generative Modeling| Diffusion Models

DMs learn transformations that transport the data to noise (the prior) and denoise it back to the input



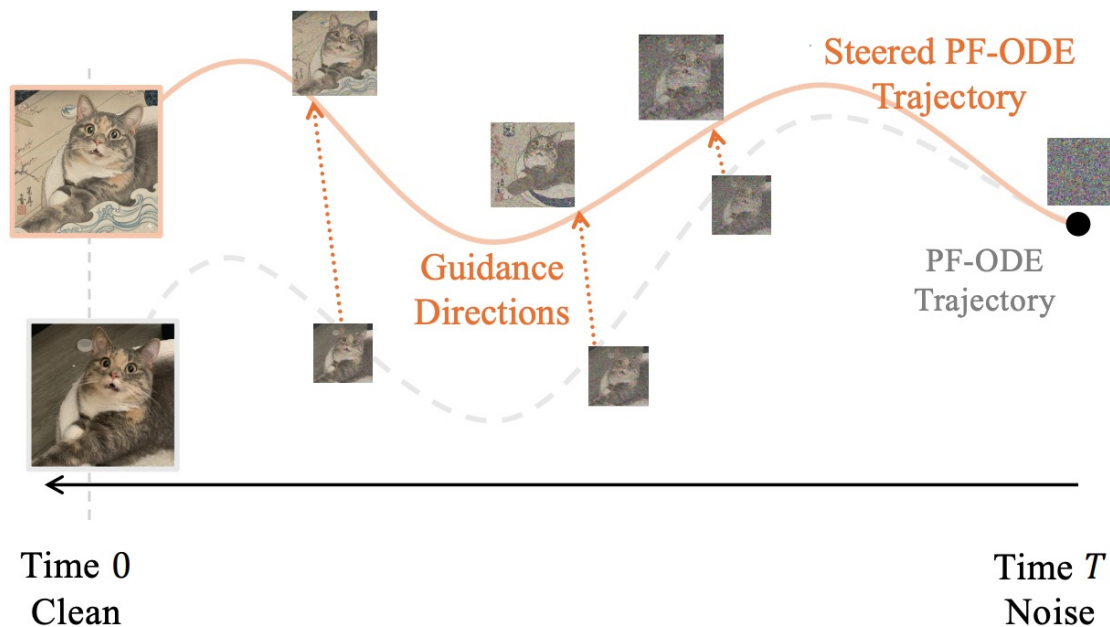
Lai (2025). The principles of diffusion models. *arXiv*

Generative Modeling| Diffusion Models



Lai (2025). The principles of diffusion models. *arXiv*

Generation is modeled as denoising the data according to a numerical simulator (ODE/SDE)



Generative Modeling| Diffusion Models



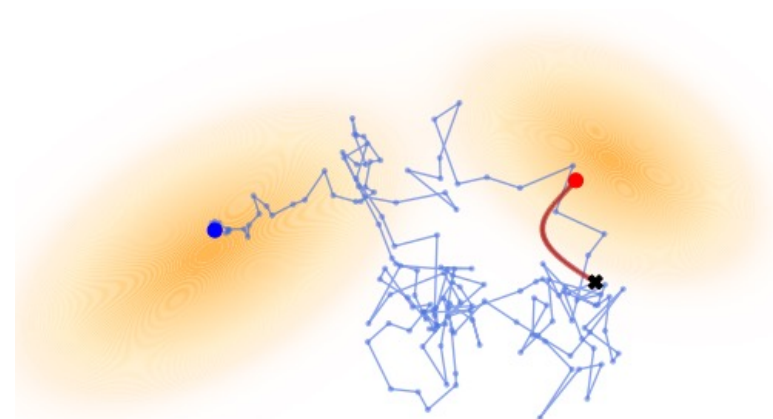
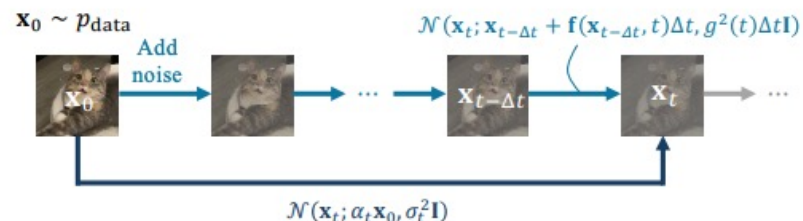
Lai (2025). The principles of diffusion models. *arXiv*

Diffusion models are generators. There are a variety of samplers to generate samples from these models.

Some well-established samplers include:

Euler-Maruyama: draw an initial value from the prior distribution. Numerically solve the SDE backward in time from $t=0$ to $t=T$. This solver provides discrete updates

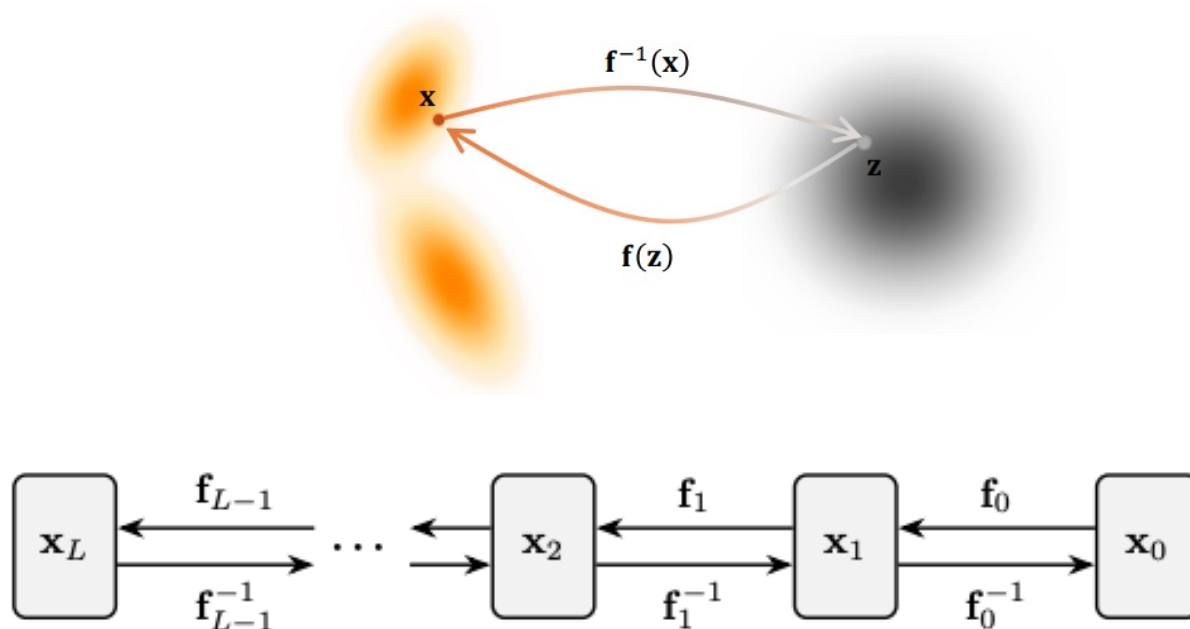
$$\mathbf{x}_{t-\Delta t} \leftarrow \mathbf{x}_t - \left[\mathbf{f}(\mathbf{x}_t, t) - g^2(t) \mathbf{s}_{\phi \times}(\mathbf{x}_t, t) \right] \Delta t + g(t) \sqrt{\Delta t} \cdot \epsilon,$$



Sampling from a Diffusion Model for inference

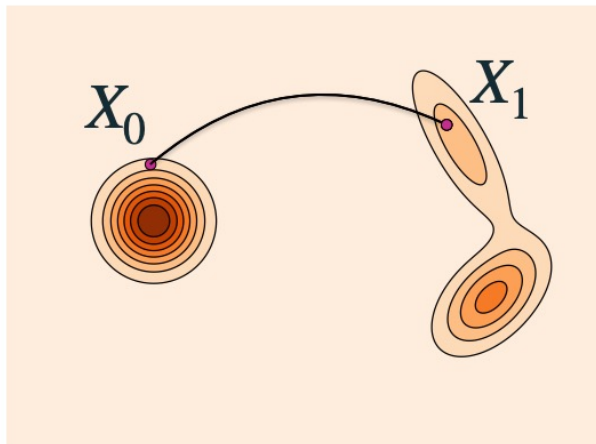
Generative Modeling| Flow Matching

Flow matching directly learns a velocity field to transport samples between distributions by applying a series of ***invertible transformations, f*** , to a simple base distribution.

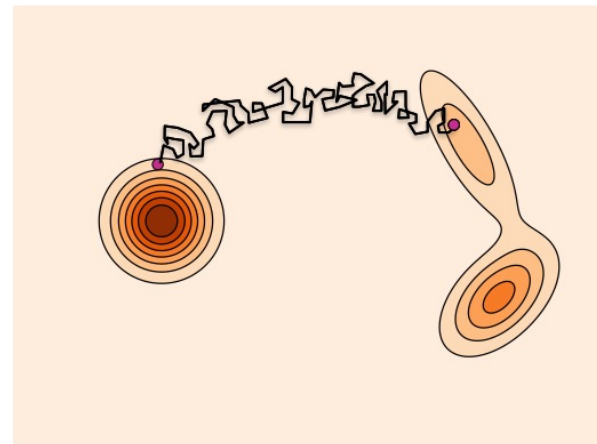


Generative Modeling| Flow Matching

- DMs learn a continuous-time flow, either stochastic or deterministic, that transports a simple Gaussian prior sample $\epsilon \sim$ prior to a data-like sample from p_{data} .
- The Flow Matching (FM) framework generalizes it to learn a flow between two arbitrary fixed endpoint distributions: a source distribution p_{src} and a target distribution p_{tgt} , both assumed to be easy to sample from



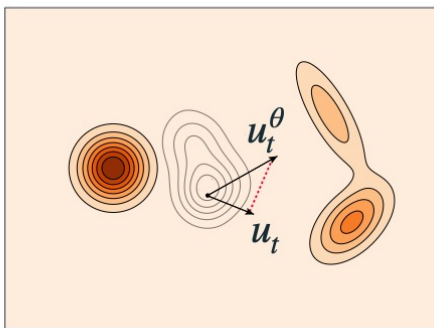
Flow



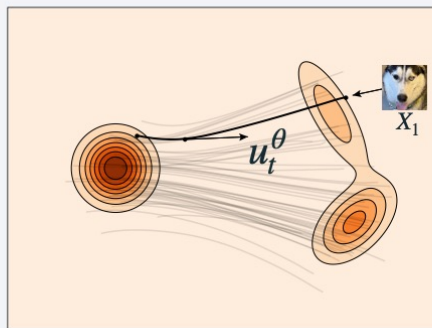
Diffusion

Generative Modeling| Flow Matching

Flow matching directly learns a velocity field to transport samples between distributions



Train a velocity
generating p_t with
 $p_0 = p$ and $p_1 = q$



Sample
from $X_0 \sim p$

Facebook Research provides a comprehensive library for FM
https://github.com/facebookresearch/flow_matching

Generative Modeling| Diffusion model vs Flow Matching

Diffusion models learn a generative model of the data by transforming the data to and from a Gaussian noise distribution, whereas FM supports mapping between any two distribution



Lai (2025). The principles of diffusion models. *arXiv*

Aspect	Diffusion Model	General FM
Source dist. p_{src}	Gaussian prior	Any
Target dist. p_{tgt}	Data distribution	Any
Latent dist. $\pi(\mathbf{z})$	p_{data}	See Section 5.3.2
Cond. dist. $p_t(\mathbf{x}_t \mathbf{z})$	$\mathcal{N}(\mathbf{x}_t; \alpha_t\mathbf{x}_0, \sigma_t^2\mathbf{I})$	See Section 5.3.2
Marginal dist. $p_t(\mathbf{x}_t)$	$\int p_t(\mathbf{x}_t \mathbf{x}_0)p_{\text{data}}(\mathbf{x}_0) \, d\mathbf{x}_0$	$\int p_t(\mathbf{x}_t \mathbf{z})\pi(\mathbf{z}) \, d\mathbf{z}$
Cond. velocity $\mathbf{v}_t(\mathbf{x} \mathbf{z})$	$f(t)\mathbf{x} - \frac{1}{2}g^2(t)\nabla \log p_t(\mathbf{x} \mathbf{x}_0)$	See Section 5.3.2
Marginal velocity $\mathbf{v}_t(\mathbf{x})$	$f(t)\mathbf{x} - \frac{1}{2}g^2(t)\nabla \log p_t(\mathbf{x})$	See Equation (5.2.10)
Learning objective	$\mathcal{L}_{\text{SM}} = \mathcal{L}_{\text{DSM}} + C$	$\mathcal{L}_{\text{FM}} = \mathcal{L}_{\text{CFM}} + C$
Underlying Rule	Fokker-Planck / Continuity Equation	

Additional useful resources on Diffusion Models and Flow Matching



Lai (2025). The principles of diffusion models. *arXiv*

Google DeepMind

From Diffusion Models to Schrödinger Bridges

Generative Modeling meets Optimal Transport

Arnaud Doucet

Special thanks to Valentin De Bortoli

Breiman Lecture - NeurIPS

12th December 2024

NeurIPS2024 tutorial

Flow Matching Tutorial



Heli Ben-Hamu Ricky T. Q. Chen Yaron Lipman

Meta

Flow Matching tutorial by Meta

Introduction to Flow Matching and Diffusion Models 2026

MIT Course 6.S184: Generative AI with Stochastic Differential Equations

Diffusion and flow models are the cutting edge generative AI methods for images, videos, and many other data types. This course offers a comprehensive introduction for students and researchers seeking a deeper understanding of these models. Lectures will teach the core mathematical concepts necessary to understand diffusion models, including stochastic differential equations and the Fokker-Planck equation, and will provide a step-by-step explanation of the components of each model. Labs will accompany each lecture allowing students to gain practical, hands-on experience with the concepts learned in a guided manner. At the end of the class, students will have built a latent diffusion model from scratch – and along the way, will have gained hands-on experience with the mathematical toolbox of stochastic analysis that is useful in many other fields. This course is ideal for those who want to explore the frontiers of generative AI through a mix of theory and practice. We recommend some prior experience with probability theory and deep learning.

Teaching materials from MIT

<https://diffusion.csail.mit.edu/2026/index.html>